DiSC: Backpressure Mitigation In Multi-tier Applications With Distributed Shared Connection

Brice Ekane and Djob Mvondo, *Univ. Rennes, Inria, CNRS, IRISA, France;*
Renaud Lachaize, *Univ. Grenoble Alpes, CNRS, Inria, Grenoble INP, LIG, 38000 Grenoble, France;*
Yérom-David Bromberg, *Univ. Rennes, Inria, CNRS, IRISA, France;*
Alain Tchana, *Univ. Grenoble Alpes, CNRS, Inria, Grenoble INP, LIG, 38000 Grenoble, France;*
Daniel Hagimont, *IRIT, Université de Toulouse, CNRS, Toulouse INP, UT3 Toulouse, France*

https://www.usenix.org/conference/nsdi25/presentation/ekane

This paper is included in the
Proceedings of the 22nd USENIX Symposium on
Networked Systems Design and Implementation.

April 28–30, 2025 • Philadelphia, PA, USA

978-1-939133-46-5

Open access to the Proceedings of the
22nd USENIX Symposium on Networked
Systems Design and Implementation
is sponsored by

DISC: Backpressure Mitigation in Multi-tier Applications with Distributed Shared Connection

Brice Ekane*, Djob Mvondo*, Renaud Lachaize†,
Yérom-David Bromberg*, Alain Tchana†, Daniel Hagimont‡
*Univ. Rennes, Inria, CNRS, IRISA, France
†Univ. Grenoble Alpes, CNRS, Inria, Grenoble INP, LIG, 38000 Grenoble, France
‡IRIT, Université de Toulouse, CNRS, Toulouse INP, UT3 Toulouse, France

Abstract

Most data-center applications are based on a multi-tier architecture, involving either coarse-grained software components (e.g., traditional 3-tier web applications) or fine-grained ones (e.g., microservices). Such applications are prone to the backpressure problem, which introduces a strong performance coupling between tiers, thus degrading scalability and resource consumption. This problem is due to the fact that, on the response path towards the initial client, a significant fraction of the payloads in the messages exchanged between tiers correspond to "final" data that are simply relayed (i.e., without further modifications) from a backend tier such as a database. This traffic results in additional pressure on the intermediate and frontend tiers.

To address this problem, we introduce DISC, a system allowing several tiers within a multi-tier chain to jointly act as endpoints of the same TCP connection. This enables the selective bypass of one or several tiers on the response path. Unlike existing solutions, DISC is (1) flexible — it accommodates arbitrary multi-tier topologies and heterogeneous application-level protocols, (2) fine-grained — it allows multiple tiers to be involved in the generation and emission of a given response message (e.g., to decouple the network path of the response headers and footers from the path of the response body), (3) and non-intrusive — it requires only minor and localized/modular modifications to the code base of legacy applications and is transparent for external clients. Evaluation results with several micro- and macro-benchmarks show that DISC can reduce the cumulative CPU load on servers by up to 41.5%, decrease the average and tail latencies respectively by up to 74.1% and 5.71×, and also improve the request rate by up to 45%.

1 Introduction

For reasons of modularity and, above all, scalability, Internet services are built in the form of an assembly of components in an architecture formerly referred to as *multi-tier* or more recently as *microservices*. In such an architecture, components are loosely coupled and communicate through messages using protocols such as HTTP, IMAP, gRPC, etc. In the remainder of this paper, we use the terms *"tier"* and *"service"* interchangeably, as our contribution is agnostic to the number and granularity of these components.

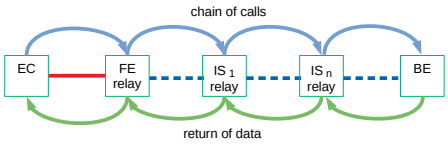

Figure 1: A schematic view of a chain of tiers. Tier BE returns data which is relayed by tiers IS_i, up to FE toward client EC.

Tier *independence* is a fundamental property that makes the popularity of multi-tier architectures. However, this independence is often compromised regarding performance, making it challenging to scale each tier independently. Our contribution aims at reducing redundant data transfers, which are a major impediment to *performance independence* of tiers. As illustrated in Fig. 1, there are redundant data transfers among different tiers involved; a request initiated by an External Client (EC) is received by a first-tier, known as the FrontEnd (FE). FE may require a call to another tier, thus forming a chain of invocations traversing Intermediate Servers (IS) up to a final tier named the BackEnd (BE). Such a chain can be noted as the following: $EC \leftrightarrow FE \leftrightarrow IS_1 \leftrightarrow \ldots \leftrightarrow IS_n \leftrightarrow BE$. In this chain, when BE produces a reply that must be returned to EC without further (read/write) processing by the IS and FE tiers, it still must go up the chain in the reverse order of request calls, generating redundant communications. We refer to such replies as *final data*. An example of final data is the mail (text and attached files) in a (multi-tier) webmail application such as Zimbra [7]. Even if IS and FE tiers only relay the final data, they undergo a load proportional to the activity of BE. Consequently, the performance of IS and FE is strongly coupled with the activity of BE. This is known as the *backpressure* problem [15]. Performance dependency among tiers makes it challenging to manage the scalability of the global application

because of the risk of the FE being the bottleneck (see §2).

For more than 20 years, numerous research works [2, 11, 14, 17, 18, 20, 21, 24, 27] have regularly proposed techniques to overcome this problem, yet with limitations. A first limitation is that these works mainly target 2-tier architectures composed of a load balancer distributing the load between a set of backends whereas applications may include much longer chains. For instance, JEE clustered architectures may include many tiers such as: an HTTP load balancer, a web server, a servlet container, an EJB server, a database load balancer, and a database. We also observed such long chains in the Zimbra clustered email server. The same observation applies to modern applications deployed in data centers, such as LAMP/MEAN stacks and microservices, which rely increasingly on long chains of existing tiers for scalability purposes.

A second limitation of all existing approaches is that they rely systematically on the full transfer of the connection from the EC to the BE, which is analogous to a migration. These solutions are radical as they cannot be generalized; they bring two key constraints. (1) Transferring the connection from a tier T_1 to a tier T_2 requires that the two tiers implement both the same communication protocol (e.g., HTTP) and API. This is why previous works have been limited to a 2-tier architecture. As is generally the case for the scalability of storage services [17], these two tiers comprise only a load balancer and a backend stage. Conversely, a complex multi-tier architecture, such as Zimbra, must generally use different protocols and APIs among tiers and is therefore incompatible with the full transfer principle. (2) Transferring the connection from a tier T_1 to a tier T_2 requires that T_1 can work correctly and efficiently without receiving (after connection transfer) requests/replies, which is only possible for simple and coarse-grained load balancing policies. Modern load balancers are not compatible with such connection transfers as they leverage the metadata included in the headers/footers of requests and replies. For example, to select the target backend, HAProxy [4] uses attributes of the request header and NGINX [5] tracks the return time of the reply header.

In a nutshell, the ideal solution to the backpressure problem is to remove redundant communications of final data. To this end, the following conditions must be fulfilled: (1) allow final data to bypass tiers when required, (2) support heterogeneous protocols (e.g., combining HTTP and IMAP) and APIs in the tier chain (which is not compatible with the full transfer principle), (3) guarantee the correct operation of multi-tier architectures (e.g., prevent inconsistencies when tiers are bypassed), (4) do not require the modification of clients, (5) support secure communication channels (e.g., TLS).

This article presents DISC, the first approach (to the best of our knowledge) that meets all of the above 5 requirements at once. DISC bypasses IS and FE adequately by allowing a BE to directly send the final data to an EC. We design DISC in such a way that the bypass can be applied to any sub-part of a chain of arbitrary depth. To operate seamlessly, DISC disaggregates the reply, which is decomposed into two parts: (i) metadata (i.e., headers/footers), and (ii) payload. DISC lets metadata (generally limited in size) follow the reply path along the chain whereas payloads (generally substantial in size) are subject to bypass IS and FE. DISC can be implemented atop any transport protocol; we chose TCP for our prototype.

Designing DISC has raised 5 major challenges. (C_1) ***TCP sequence number management.*** Since a tier can return payloads directly on the TCP connection established between two other tiers, it is necessary to coordinate the emission of the sequence numbers so that it is transparent to those tiers. DISC allocates and translates the sequence numbers to take into account distributed emitters. (C_2) ***Management of TCP acknowledgments.*** Since the data transmitted comes from different tiers, acknowledgments received from the destination must be propagated to emitters so that they can move forward in the payload transmission process. DISC routes acknowledgments towards the concerned emitter. (C_3) ***Integration into protocol stacks.*** To avoid modifications of the Linux kernel protocol stacks, DISC extensively relies on BPF. (C_4) ***TLS support.*** Many communications over the Internet are secured using TLS. Therefore, all tiers involved in a chain must be able to return encrypted payloads in the same TLS session. To this end, DISC uses a serialization mechanism to ship TLS sessions among emitters. (C_5) ***Modification of applications.*** To minimize the impact of DISC on the code base of the different tiers, we present a pattern allowing to systematically modify applications.

We evaluated DISC using different representative multi-tier and microservice-based applications, in particular: SpecWeb [13], SpecMail [12], Train Ticket [22] and DeathStarBench Social Media [15]. The quantitative results show that DISC can significantly reduce the propagation of pressure in a chain of tiers, thus reducing CPU load on FE and IS but adding CPU load on the BE. Globally, with DISC, the cumulative load on servers is reduced by up to 41.5% and the supported request rate is improved by up to 45%. DISC has no impact on mean latency and can improve tail latency significantly. For instance, on a micro-benchmark with ten IS, DISC reduces tail latencies down to 5.71×. The reduction of the load on the bypassed tiers makes it possible to ensure greater independence of the tiers from the point of view of performance and scalability. With DISC, saturation occurs first at BE up to FE (thus promoting service scalability), whereas without DISC, it is the inverse; saturation occurs first at FE.

The remainder of the article is structured as follows. Section 2 details the motivations for DISC and its position w.r.t. previous works. Section 3 present the main principles of our contribution. Section 4 reports the results of evaluations. We conclude in Section 5. Additional details about the implementation and the evaluation are provided in appendices.

2 Motivation and previous works

The architectural pattern of two common Internet applications, e-commerce web applications and email services (respectively illustrated by the SpecWeb [13] and SpecMail [12] benchmarks), are described in §2.1, and, in §2.2, we evaluate the impact of their inherent redundant communications on server load and application scalability. Finally, we position DISC with respect to state of the art solutions in §2.3.

2.1 Motivating use cases

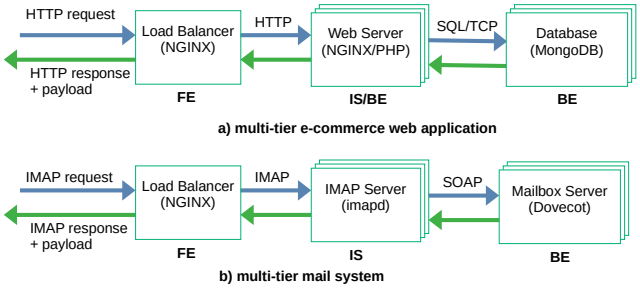

Figure 2: Two typical N-tier applications: e-commerce (SPECweb) and mailing system (SPECmail).

Coarse-grained multi-tier applications remain very popular in many production contexts. For example, according to a study [26], the WordPress content management system [10] is used by 43% of the websites on the Internet. To explain the motivation behind DISC, we use two multi-tier benchmarks due to their simplicity. These two applications involve different protocols such as HTTP, IMAP or SOAP. Notice that, we also demonstrate later how DISC can be beneficial for microservices-based applications (see §4.5 and Appendix B).

E-commerce application. Fig. 2-a depicts a common setup of a 3-tier architecture for an e-commerce application. In this architecture, the FE is a load balancer (NGINX) distributing requests received from EC to a set of web servers. These web servers (NGINX/PHP) can serve as BE, and return static pages (e.g., images). They can also act as IS when executing PHP code that requires database access; in this case, a request is sent to the database (MongoDB), which can in turn serve as BE, and can return dynamic content as final data (e.g., a JSON collection). In our experiments, we used the e-commerce application (and load injector) from SpecWeb [13].

Email serving. Fig. 2-b illustrates a possible deployment setup of a 3-tier architecture for the Zimbra email system. As in the previous application, the FE is also a load balancer (NGINX). It distributes requests received from EC to a set of IMAP servers (imapd). These IMAP servers act as IS, and send requests to Mailbox servers where client emails and folders are managed. The Mailbox servers are implemented using Dovecot [3], and serve as BE. In our experiments, we used SpecMail [12], which provides a load injector.

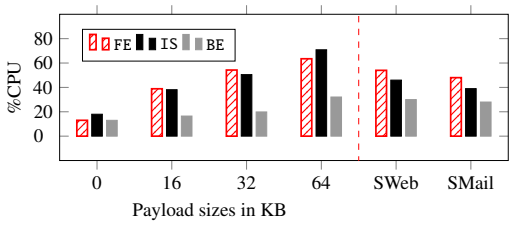

Figure 3: CPU load across each tier (vanilla setting).

2.2 Problem assessment

Final data returned by the BE must go up the chain in the reverse order from tier to tier until it reaches the EC.

Backpressure. To highlight the problem, we study the communications in a traditional multi-tier design (hereafter named *"vanilla"*). We assess the impact, in terms of CPU consumption, of relaying final data on each tier of a 3-tier architecture. We evaluate three applications: a micro-benchmark made of a chain of 3 NGINX servers, SpecWeb, and SpecMail. For the micro-benchmark, the EC submits a constant load to the first tier of the chain (FE), while the payload size of the final data returned by the BE varies from 0KB to 64KB (all the returned data is final). Conversely, for SpecWeb and SpecMail, the amount of final payloads and their sizes vary according to the benchmark instantiated. The detailed setups for these evaluations are provided in §4. At the left of the dashed line on Fig. 3, we show the CPU load on each tier of the chain (i.e., FE, IS, and BE) for the micro-benchmark. We observe a higher load on the FE and IS tiers compared to the BE tier. Furthermore, as the size of the payload returned by the BE increases, the CPU load logically increases on the BE, but it rises more significantly on the FE and IS tiers. This clearly illustrates the issue of backpressure, whereby the increased load on the BE propagates to the FE and IS tiers. Additionally, a payload size of 0 provides an estimate of the CPU load on FE and IS tiers if the payloads were directly returned to EC by the BE, thus bypassing FE and IS tiers. At the right of the dashed line on Fig. 3, we show the results for SpecWeb and SpecMail. We observe similar trends compared to the micro-benchmark.

Scalability. This backpressure problem has a significant impact on the application's scalability. To demonstrate this, we consider the same micro benchmark based on an FE-BE architecture. A load injector (EC) generates requests at a growing rate (+2K req/s every 3 seconds). We start each tier with 2 CPU cores allocated and we double the number of cores of a

tier (FE or BE) when it becomes saturated (vertical scalability). The results are shown on Fig. 4-top (Fig. 4-bottom displays the results achieved with DISC, discussed later in §4.2).

As shown at the left of the solid line on Fig. 4-top, the FE (grey curve) is the first tier to saturate due to the load on the BE that has propagated to its preceding tier. This saturation impacts the average latency measured from the client (dashed curve). At the right side of the solid line on Fig. 4-top, we added 2 cores to the FE, thus allowing the application to scale.

The goal of DISC is to ensure that an increase of the load on the BE (which returns final data to EC) does not propagate to the preceding tiers, and therefore does not lead to a saturation of the FE first, requiring the allocation of additional resources on these tiers. Ideally, an increase of the load on the BE should only lead to an increase of allocated resources on the BE.

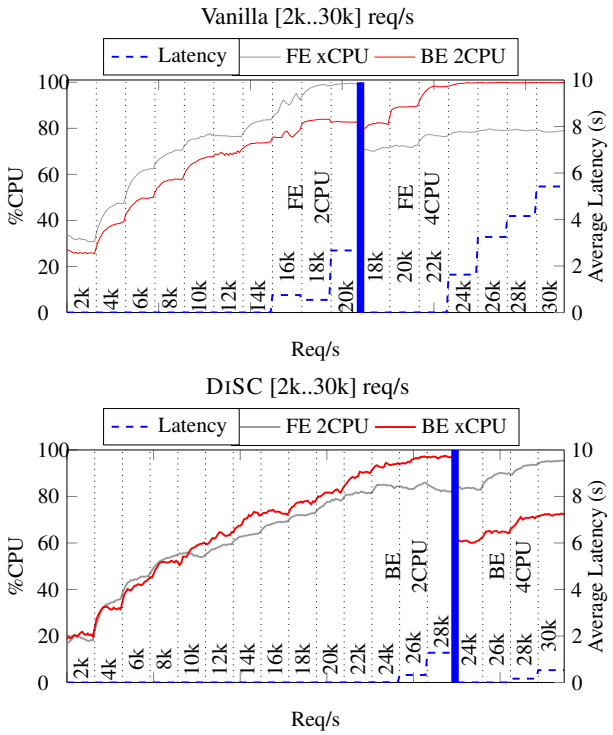

Figure 4: CPU load on the FE and BE and overall latency when increasing the request rate. Vanilla (top) vs DISC (bottom).

2.3 Shortcomings of existing solutions

Over the last decades, several works have partially addressed the backpressure problem [2,17,20,21,27]. They are designed around the *connection handoff* concept [11,18], i.e., the migration of connections between servers. However, existing handoff-based solutions have three major limitations: (i) short depth path, (ii) API lock-in, and (iii) side effects of shortcuts. We also discuss the limitations of QUIC [23], which could falsely appear to be an alternative to DISC. Given our focus on compatibility with existing datacenter applications and infrastructures, we do not discuss data transfer optimizations needing invasive software or hardware modifications [28,29].

Short depth path. The two most recent contributions, CRAB [20] and Prism [17] only support a 2-tier architecture; the FE acts as a load balancer to distribute requests to several BE. As a result, they can only be used to bypass a single server. Moreover, most common n-tier architectures, like those described in §2.1, are based on 3 tiers or more. DISC is independent of the number of tiers, and allows bypassing more than one tier. Besides, unlike Prism, DISC does not require specific hardware support (smart switches).

Application protocol/interface lock-in. Existing solutions rely on connection migration and therefore they require all tiers (FE and BE) to implement the same protocol (e.g., HTTP) and API, which is a strong constraint. As mentioned in §2.1, a multi-tier architecture may involve a combination of different protocols (e.g., HTTP and IMAP). DISC is independent of the application-level protocols and the APIs being used.

Side effects of shortcuts. Connection migration fully bypasses the FE, for both onward and backward traffic. A full bypass may have a significant impact on a FE that acts as a load balancer, which can lead to malfunctioning. This comes from two main reasons: (1) metadata attached to the final data (e.g., header, or footer) may contain information provided by the BE required by the FE to adjust its behavior a *posteriori* (e.g., for feedback-driven load balancing leveraging the request backlog size of each BE). (2) More importantly, receiving responses is an invariant of the execution logic of a multi-tier application. Completely bypassing a tier requires in-depth modifications of its code, which breaks its genericity.

QUIC limitations. QUIC [23] is a transport protocol recently standardized by IETF, implemented in user level and based on UDP. QUIC provides multiple streams within a connection (similarly to HTTP/2, although QUIC does not suffer from head-of-line blocking). It relies on a Connection ID to identify connections instead of IP/port, thus easing connection migration across IP address changes. This way, it provides a mechanism for client-side connection migration, e.g., when a client changes its network connectivity (thus its IP and port) on network handover. Thus, QUIC provides a mechanism equivalent to TCP hand-off. As stated above, hand-off solutions require all servers to implement the same protocol and API (e.g., HTTP) on top of the transport protocol (TCP or QUIC). Besides, some works [25] investigated server-side connection migration in QUIC, especially for migrating to a closer server in an edge computing context. However, such an extension requires modifications both on the server and client sides of QUIC (while DISC is transparent for the client).

QDSR [30] brings the benefits of DSR to QUIC-based L7 load balancers and offloads the responses for distinct streams to different backend servers. This approach also suffers from the limitations discussed in the three previous paragraphs.

3 Our approach: DISC

To operate shortcuts, DISC relies on a new protocol, DISC-PROT. This protocol adds header information to the request and response messages on top of the underlying L4 protocol (TCP). It enables any node in the chain to indicate in a request, which transits through the forward path, that it can be later bypassed, i.e., that it can be skipped in the backward path for the transmission of the final response's payload.

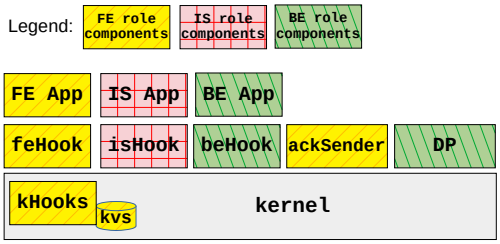

Figure 5: Software architecture of DISC on a server, with components used according to the tier role.

3.1 DISC architecture

A cloud service, invoked by an external client, is made of a set of tiers (hosted by servers in datacenters) structured as a directed acyclic graph that is traversed through request-response invocations following a chain of calls.

Roles. In a chain, depending on whether it is bypassed or not, a tier T_i can play one or the other of the following roles: (i) BE, (ii) EC, (iii) IS, and (iv) FE. T_i acts as a BE (*i.e., it has a backend behavior*) if it sends the final response. T_i acts as an EC (*i.e., it has a client behavior*) if it emits the initial request, and hence is the target of the final response. Usually such a role is taken by the external client, but nothing prevents a tier from having this role if the shortcut occurs in the middle of the chain. T_i acts as an IS (*i.e., it has an intermediary tier/service behavior*) if it is bypassed in the chain, while still receiving the metadata of the final response (without its payload). Finally, T_i acts as FE (*i.e., it has a frontend behavior*) if it is directly connected to the one that acts as an EC. In this particular case, it will ask the first encountered previous BE to send the final response to EC.

Components. DISC is implemented as a kernel extension, and is bundled with several additional components in both the kernel and user space. These components enter in action according to the role played by tiers, i.e., FE, IS, BE (see Fig. 5). Note that to act as an EC, DISC is not required as it is transparent — external clients do not need DISC. However, DISC must be deployed on all servers of the datacenter to perform shortcuts. From a kernel-space perspective, DISC includes two kernel modules: (i) a key-value store, named *kvs*, to register information about in-progress requests, and (ii) a kernel hook called *kHook* for treating incoming/outgoing L4 packets (e.g., connection establishment). From a user-space perspective, DISC provides different hooks (i.e., *feHooks*, *isHook* and *beHook*) to act at the L7 layer (such hooks allow adapting applications) and two user-level modules, noted *ackSender* and *DP*. The former manages the acknowledgment of L4 packets, and the latter takes care of sending the payload to be sent directly from BE to EC (i.e., it performs the shortcut).

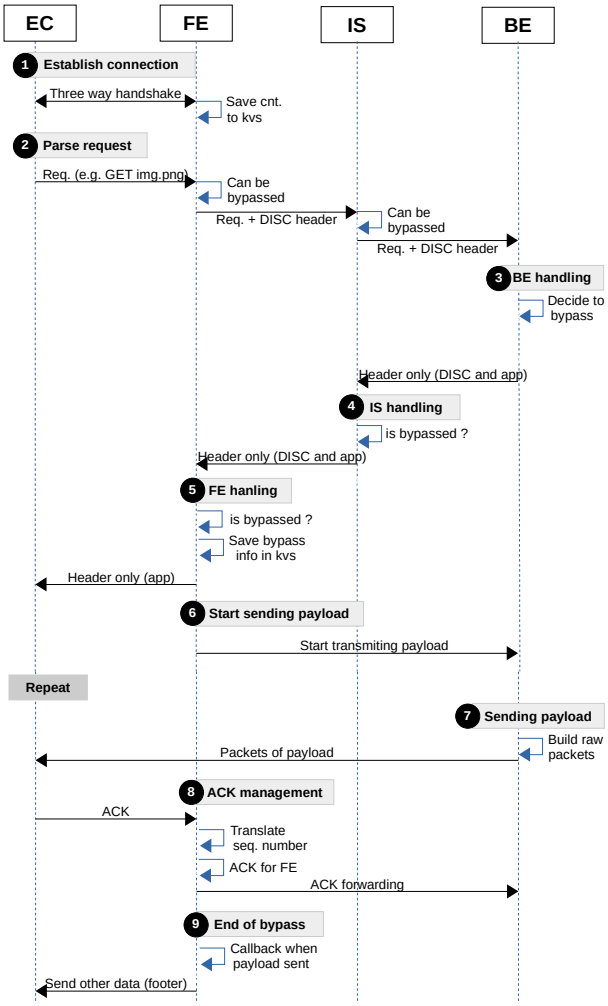

Figure 6: End-to-end coordination with DISC.

DISC shortcut mechanisms. A shortcut is the result of a coordination among FE, IS, BE and EC in 9 phases. Fig. 6 illustrates the different phases in terms of packet sequence

diagram for a scenario where BE sends the final response on the initial connection between FE and EC. The 9 phases are:

(1) Establish connection. When EC connects with FE, kHook intercepts three handshake messages and when the connection is established, it creates a key in kvs that identifies the TCP session. This key is associated with a record `<source IP, source port, ...>` that includes all information required for sending packets to EC and handling acks. Notice that all packets received on that connection are intercepted by kHook, especially acks (detailed in §3.3). As we show in §4, this interception does not impact request latency.

(2) Parse request. When EC sends a request, FE can indicate (*via* feHook, which adds an option in the DISC-PROT protocol) that it can be bypassed for the returned payload (see details in §3.2). Here, feHook is an adaptation in the L7 layer, where any policy can be applied for accepting or not the bypass, depending on the need for that tier to have or not access to the returned payload. Notice here that if FE is bypassed regarding the payload, it will anyway receive the application response header (e.g., IMAP header) and footer (e.g., IMAP footer) that can be used for internal processing. Then, FE forwards the request to IS, which behaves similarly (with its isHook) and forwards in turn the request to BE.

(3) BE handling. BE uses beHook to enable the current tier to decide whether the payload is eligible to bypass the previous tiers. If bypass is enabled, the payload is sent to the DP module, instead of the TCP connection with the preceding IS. At this point, DP generates an identifier (ids) that designates the payload in DP. Then, beHook builds a DISC-PROT header that includes the following options: a flag (*bypass*) saying that the payload is subject to bypass, the previous DP identifier (ids), the size of the payload and the IP address of the BE (allowing the FE to later request from the DP the emission of the payload, thus bypassing other tiers, FE and IS). Then, the BE application-level protocol builds its header, but with a payload size set to zero. Both headers (DISC-PROT and application-level) are included in the response message.

(4) IS handling. On IS, the response message is received and isHook checks the bypass option to skip the reading of the payload if bypassed. IS then handles the response normally and sends the response (including the DISC-PROT header) to its preceding FE, except it omits the payload if bypassed.

(5) FE handling. Like on IS, feHook checks the bypass option to read or not the payload. If the FE is bypassed, then feHook extracts all information about the bypass from the (DISC-PROT protocol) response header and stores them in the kvs entry associated with the associated initial connection. This information includes the IP of BE, the ids of the payload and its size, that will be used later to handle acks. Thus, FE can send the application-level response header back to EC.

(6) Sending the payload on FE. On FE, feHook then invokes (remotely) the DP on BE (it has its IP address) asking the emission of the payload identified with ids. This invocation provides to DP all information required to spoof FE (including TCP sequence numbers to be used), thus sending the payload on the initial connection.

(7) Sending the payload in DP. DP builds raw packets and sends them on the initial connection towards the EC. From that time, FE will receive acks for these packets (as DISC does not modify the Client). To allow the DP to continue emitting the payload (increase its emission window), received acks on FE (related to the payload) are forwarded to the DP.

(8) ACK management. All received acks on FE are intercepted by kHook (the kernel module), analyzed and handled. kHook is responsible for ensuring consistency of TCP sequence numbers. Indeed, the TCP stack on FE is not aware of packets sent by DP, thus kHook has to apply translations on sequence numbers. Also, acks received from EC may acknowledge packets emitted by the TCP stack on FE, or by DP. Acks concerning FE's stack have to be translated and acks concerning DP are forwarded to DP. Every ack concerning DP is stored in kvs and the ackSender process (started on FE) polls kvs to extract acks and forward them to DP. TCP sequence number management is detailed in §3.3.

(9) End of bypass. An application-level protocol may include a header and a footer. This is for instance the case with the IMAP protocol. Therefore FE, which sent on the initial connection its response header and delegated the emission of the payload to BE, may need to send a footer after the payload. In order to provide a means for that to the feHook, we provide an API for registering a callback function. This callback function is called when kHook observes from received acks that the full payload was received by the Client. feHook can then prepare the footer and register a callback function that will send the footer on the initial connection when the payload was totally sent. Notice here that we did not want to address this footer need through a synchronous invocation of the DP (which would have returned when the payload is sent), as it would have affected the threading model of FE. With this callback-based mechanism, the handling of footers is kept asynchronous (without blocking a thread). With DISC, we observe that the capacity to emit on the initial connection is temporarily delegated to BE and gets back to FE for sending the footer or sending the response header of the next request.

3.2 Instantiation

The previous section gave the general principle of DISC in a scenario with a unique shortcut, where a payload sent by BE bypasses IS and FE and is sent directly to EC. Now, we consider a more general example with 4 tiers chained as follow: $T_0 \leftrightarrow T_1 \leftrightarrow T_2 \leftrightarrow T_3$, as depicted in Fig. 7, and with two shortcuts at once; T_0 and T_2 can be bypassed but not T_1. The DISC-PROT protocol includes two options for managing shortcuts in its header: `NB_TIER` and `BYPASSED_TIERS`. The former is a counter initialized to 0 on T_0, incremented each time a request is forwarded to the next hop and decremented each time a response is returned to the preceding hop. It

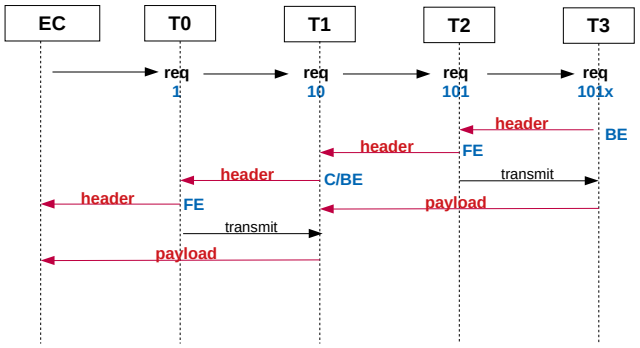

Figure 7: Concrete example of DISC with an external client invoking a chain of 4 tiers. Here, tiers T_0 and T_2 are bypassed.

permits a tier to know its place in the path and to adequately set the bit of the bitmap `BYPASSED_TIER`, indicating whether or not it wants to be bypassed.

Roles. T_3 acts as BE. Bypassing is possible if T_3 wants it for the current payload according to its policy (e.g., payload size greater than 20KB) and if the next tier (i.e., T_2) agrees to be bypassed. To perform a bypass, T_3 splits the final response into two responses and sends (i) one response without payload, i.e., a header and/or a footer, to T_2 to follow the backward path (lightweight forward), and (ii), one response that includes the payload to the local DP (shortcut). Otherwise, if there is no bypass, the final response is sent to T_2 (heavyweight forward).

Regarding T_1 or T_2, they can play different roles depending on whether they accept bypassing. If it does not agree to be bypassed (e.g., T_1), it receives a response that includes a payload. Hence, it plays simultaneously both EC and BE roles. Particularly, it acts as BE from the perspective of the next tier (e.g., T_0), and it acts as EC from the perspective of the previous tier (e.g., T_2). Otherwise, if it agrees to be bypassed (e.g., T_2), it receives a response without payload as it was checked by the tier that sent that response. Then, in this case, if the next tier (e.g., T_1) agrees to be bypassed, it plays the IS role (lightweight forward), otherwise it plays the FE role. In this case, it (e.g., T_2) remotely asks the DP of the first previous BE encountered (e.g., T_3) which held the payload to send it on the connection with the next tier (e.g., T_1). In this latter case, the next tier will receive a final response, acting as EC and BE as seen earlier.

Regarding T_0, if it receives a response with a payload (it plays an EC role), it simply sends it on the initial connection. Otherwise, if it receives a response without payload, it plays the FE role. Accordingly, it remotely asks the DP of the first previous BE encountered (e.g., T_1) that held the payload to send it on the connection of either the next tier (and so on) or the external client if the end of the chain is reached.

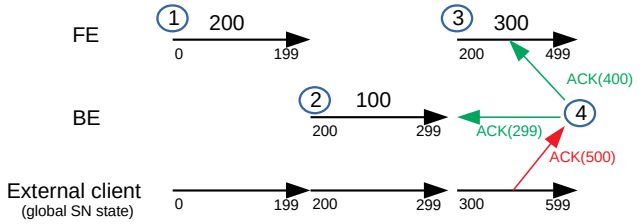

Figure 8: Coordination of TCP sequence numbers.

Bitmaps. In the scenario of Fig. 7, the final value of the bitmap is *101x* (the first bit is the leftmost one). T_3 decides to enable bypass (e.g., based on the payload size). Thus, it plays the BE role. As T_2 agrees to be bypassed (in Fig. 7, the 3rd bit of the bitmap is 1), T_3 sends a response without payload, and the payload is sent to DP. T_2 plays the FE role since T_1 does not agree to be bypassed (in Fig. 7, the 2nd bit is 0). Therefore, T_2 sends a response without payload, but it is sent from T_3 (its DP) to the T_2-T_1 connection. T_1 receives a response with payload, so it plays the EC and BE roles, and since T_0 agrees to be bypassed (in Fig. 7, the 1st bit is 1), T_1 sends a response without payload, and the payload is sent to its DP. T_0 receives a response without payload, and since its predecessor (EC) cannot be bypassed, it plays the FE role, thus sending a response without payload, but the payload is sent from T_1 (DP) on the T_0-EC connection (initial connection).

3.3 DISC atop TCP

First, we stress that DISC's design and implementation do not hinder the characteristics of applications built atop TCP. In particular, DISC is compatible with keep-alive connections, piggybacking (an ACK carries a new request), and request pipelining and does not negatively impact (or remove) TCP's congestion control mechanism. Below, we describe the main challenges introduced by DISC w.r.t. TCP. Implementation details for our DISC prototype are provided in Appendix A.

As we do not want to modify the TCP stack deployed on the servers (DISC pervasiveness), DISC needs to coordinate the TCP Sequence Number (SN) emitted by FE and BE. Indeed, EC is likely to receive TCP packets from multiple sources, such as from FE and BE, so both must send packets with consistent sequence numbers. This is illustrated in Fig. 8 where, for more clarity, we do not present sequence numbers in terms of bytes but in terms of packets. If FE sends 200 packets with SN from 0 to 199 (❶), and thereafter BE sends 100 packets (❷); then the SN for the latter should be from 200 to 299 (BE must be informed about the SN it should use). Now, if FE sends 300 additional packets (❸); from its point of view, the SN will be from 200 to 499 since it is not aware of the packet emissions from BE. Consequently, these SN must be translated to be consistent from the point of view of EC, so they will be from 300 to 599 (instead of 200 to 499).

The management of SN in DISC is implemented in kHook as follows. kHook maintains a global SN state in kvs, considering all emitted packets (from FE and BE). The invocation of BE (especially its DP) to trigger the emission of the final payload takes as an additional parameter the global SN state so that BE can send packets with consistent SN. The global SN state can be updated as we know the payload size (from the DISC-PROT response header). In Fig. 8 at emission (❷), the global SN state is 200, and the global SN state is updated to 299 after emission. Packets emitted by FE are modified by kHook to adapt their SN according to the global SN state. In Fig. 8 at emission (❸) from FE, SN from 200 to 499 are modified and become from 300 to 599. kHook maintains in kvs the emitted SN and the servers (FE or BE) which emitted them. Any packet acknowledgment (ack/sack) received by FE is intercepted by kHook. Its SN is analyzed to either send an ack/sack packet to FE 's TCP stack (and the SN is translated back) or either send an ack/sack packet to BE which emitted it (allowing to resend a packet, free its internal buffers, and adapt its emission window). In Fig. 8, an ack with SN 500 is received (❹) on the initial TCP connection. kHook, according to its internal state, modifies the SN to 400 (since 100 packets were sent by BE) so that the ack is consistent for FE. Moreover, it generates an ack packet for the preceding emission from BE with SN 299.

Last, SN for packets received from EC (e.g., requests) are all handled by the FE and will generate ack/sack towards EC. However, all payload packets emitted by the BE towards EC must include an ack with the last acknowledged received data SN. This SN (that we call the *client SN*) is passed to the DP of the BE when invoked for payload emission so that a proper ack can be added in the emitted packets.

Due to a lack of space, we only briefly discuss the behavior of DISC in the event of failures. Without DISC, in a multi-tier architecture, the failure of one tier generates delays, ultimately leading to a disruption in the connection with the external client. The application code in the tiers is responsible for implementing precise semantics in the event of such failures. Message losses are handled by TCP, thus by DISC. In the case of DISC, sending the payload to the client is delegated to the BE by the FE (recall that the headers follow the normal path). The BE (and its DP) behaves like TCP, meaning that it retransmits lost messages and will stop (and clean up) sending the payload if the connection with EC gets broken.

3.4 TLS support

The integration of TLS in our prototype relies on wolfssl [6]. The design of TLS support extends the work described in [17]. It relies on two functions that behave as serialization primitives: `wolfSSL_tls_export()` (which transforms an `SSL_SESSION object` into an `ASN1` representation) and `wolfSSL_tls_import()` (which performs the reverse operation). In our DISC prototype, feHook and DP are mutually exclusive, i.e., they run on different machines. When feHook invokes the DP to request the emission of a payload, it invokes `wolfSSL_tls_export()` and passes as a parameter the `ASN1` representation of the SSL session. DP receives this representation and then invokes `wolfSSL_tls_import()` to recreate the SSL session. The latter is then associated with the file descriptor used to emit payload chunks directly to the client. Once the full payload has been emitted, DP sends the `ASN1` representation of the SSL session back to the FE. On the FE, kHook receives this representation and restores the SSL session. Note that the encrypted chunks of the payload are buffered in DP so that they can be re-sent if a SACK or DUP ACK is received. The FE cannot emit data toward the client while the SSL session is located on the DP (until the SSL session is returned on the FE).

This main impact of DISC's TLS support is the shift of TLS processing from the FE to the BE. From the datacenter point of view, this is not an overhead but a shift, and this can moreover be seen as a benefit since the FE (which is often a load balancer) can be a bottleneck while the BE is often sharded and replicated for scalability.

3.5 Application integration

We need to modify applications that run on FE, IS, and BE tiers to implement DISC. These modifications are hereafter referred to as feHook, isHook and beHook. In this section, we give pseudo-code, which describes the algorithms of such hooks in the case of HTTP communications. The code has to be adapted to receive and send HTTP responses without payload. As depicted in §3.2, headers (of the DISC-PROT protocol) are added in requests and responses. A DISC-PROT request header indicates whether the current tier can be bypassed, and a DISC-PROT response header indicates whether the payload is present or not.

First, for IS, the program below is a prototype of the code of the IS and its adaptation (in bold). We assume the server has two socket file descriptors: `fdc` and `fds` (respectively towards the client and the server side). This code is executed when a request is received on `fdc` and has to be forwarded to `fds`.

```
1  discprot_req_hdr = receive_discprot_request_header(fdc);
2  update_discprot_request_header(&discprot_req_hdr);
3  send_discprot_request_header(fds, discprot_req_hdr);
4   http_req_hdr = receive_http_request_header (fdc);
5   send_http_request_header (fds, http_req_hdr);
6   payload = receive_http_payload (fdc);
7   send_http_payload(fds, payload);
8     ...
9  discprot_resp_hdr = receive_discprot_response_header(fds);
10 send_discprot_response_header(fdc, discprot_resp_hdr);
11  http_resp_hdr = receive_http_response_header (fds);
12  send_http_response_header(fdc, http_resp_hdr);
13 if (! bypass(discprot_resp_hdr)) {
14        payload = receive_http_payload (fds);
15        send_http_payload(fdc, payload);
16 } else {
17      // NOTHING
18 }
```

Onwards, this program receives the DISC-PROT request header, updates it as described in §3.2, and forwards it (lines

1-3). It then receives and forwards the HTTP request header and payload (lines 4-7). Backward, it receives and forwards DISC-PROT and HTTP response headers (lines 9-12). Then, depending on the information in the DISC-PROT response header (extracted by the `bypass()` function), it receives and forwards the payload if no bypass occurs (lines 13-15).

For FE, the code adaptation is very similar to the above one, with few differences. First, the DISC-PROT request header is not received from the external client but constructed (line 1). Second, when a response is received without a payload (line 17, `NOTHING` comment), FE has to request the emission of the payload by the BE. Here (line 17), FE must execute the following additional code:

```
17    register_range(discprot_resp_hdr);
18    start_transmit(discprot_resp_hdr);
```

`register_range()` registers in the kvs the SN of the packets sent by BE. `start_transmit()` remotely invokes BE (its DP) to start the emission of the payload. The `discprot_resp_hdr` structure includes all the required parameters for these functions.

Finally, for BE, the code below is a prototype of the code of BE and its adaptations (in bold):

```
1   discprot_req_hdr = receive_discprot_request_header(fdc);
2   http_req_hdr = receive_http_request_header (fdc);
3   payload = receive_http_payload (fdc);
4
5   // process request and compute response
6   // (http_response_header and payload)
7
8   discprot_resp_hdr = make_discprot_response_header(discprot_req_hdr);
9   send_discprot_response_header(fdc, discprot_resp_hdr);
10  send_http_response_header(fdc, http_resp_hdr);
11  if (bypass(discprot_resp_hdr)) {
12          dp_send_payload(fd_disc, payload);
13  } else {
14          send_http_payload(fdc, payload);
15  }
```

Onwards, this program receives the DISC-PROT, HTTP request headers, and the request payload (lines 1-3). The application processes the request, and a response is computed. Backward, a DISC-PROT response header is computed (line 8), deciding whether a bypass occurs. The DISC-PROT and HTTP response headers are returned on the `fdc` client socket (lines 9-10). Then, according to information in the DISC-PROT response header (extracted by the `bypass()` function), the payload is either sent to the local DP component[1] (line 12) on a dedicated file descriptor (`fd_disc`) if a bypass occurs or to the preceding tier (line 14) if no bypass takes place.

4 Evaluation

Testbed. We use c220g5 nodes from CloudLab [1]. Each server has 2 Intel Xeon Silver 4114 10-core CPUs at 2.20 GHz, 192GB memory, and a 10GB Intel X520-DA2 NIC. They run Debian server 11 with Linux v5.19. For CPU measurements, in order to concentrate on DISC impact on CPU consumption, each tier is executed on one CPU core unless otherwise specified.

[1] As discussed previously (§3.1), the DP buffers the payload locally and sends it to the EC when it receives a transmission request from the FE.

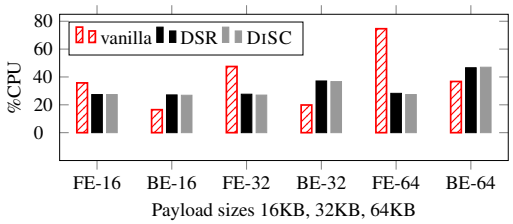

Figure 9: CPU load on each tier in `[FE-BE]` configuration.

Benchmarks. We use one micro-benchmark and four macro-benchmarks to evaluate DISC. The micro-benchmark is a simple N-tier web application that returns static contents of different sizes (16–64KB). The FE and IS tiers only relay requests/responses. We choose the size ranges based on statistics from the Internet/HTTP Archive about typical statistics for HTML pages and images (whose average sizes are respectively 32 and 48KB). It is important to note that the larger the response size, the greater the benefits of DISC.
We use wrk [16] as client, generating 750 req/sec. FE, IS, and BE are implemented with NGINX. Using this micro-benchmark, we can evaluate DISC with various numbers of IS. Concerning the macro-benchmarks, we use SpecWeb [13], SpecMail [12], Train Ticket [22], and Social Media (from the DeathStarBench suite [15]). The first two applications follow a 3-tier architecture (`FE-IS-BE`), presented in §2.1. We use them for the majority of the experiments. We simulate 700 clients. The requested payload sizes are respectively in the [1–64 KB] and [1–100 KB] ranges for SpecWeb and SpecMail. The two other macro-benchmarks, Train Ticket and Social Media, are based on the microservices paradigm. Their characteristics and setups are detailed in §4.5 and §B, respectively.
Goals. We consider five questions: (Q_1) Does DISC effectively offload load from FE and IS, thus addressing the back-pressure problem (§4.1)? (Q_2) What is the impact on scalability (§4.2)? (Q_3) What effect does DISC have on network latency as perceived by the application client (§4.3)? (Q_4) What is the impact of TLS support (§4.4)? (Q_5) What is the impact of DISC on microservices architectures (§4.5)?

We compare DISC with vanilla settings and Direct Server Return (DSR) [19], a standard TCP Handoff [11] solution implemented by many load balancers as stated in §2.3. DSR does not allow fine-grained bypass nor several-tier bypass, and is limited to a single protocol (HTTP). Consequently, we use DSR only for bypassing the FE tier, from its preceding tier which is BE in the `[FE-BE]` architecture or IS in the `[FE-IS-BE]` architecture. Note that, for the 4 macro-benchmarks, we only compare DISC against vanilla because they all are based on architectures featuring multiple protocols and one or several

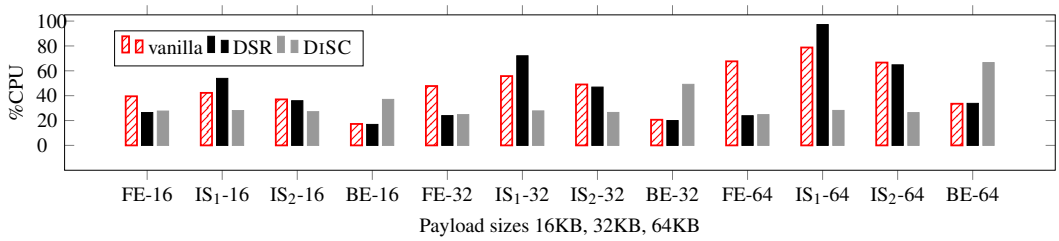

Figure 10: CPU load on each tier in [FE-IS$_1$-IS$_2$-BE] architecture.

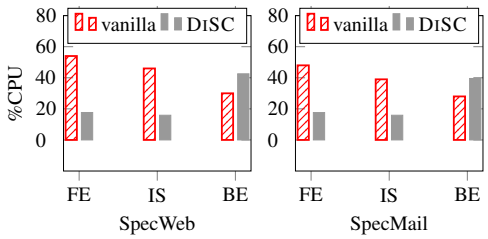

Figure 11: CPU load on SpecWeb and SpecMail tiers.

intermediate tiers.

4.1 CPU load offloading from FE and IS (Q_1)

Micro-benchmark, [FE-BE] architecture. Fig. 9 shows the CPU load for each tier while varying the size of the requested content. With vanilla, FE consumes a significant amount of CPU compared to BE. The gap increases as the content size increases. With DISC and DSR, the load is pushed on BE, where it should since most of the request processing is related to BE. Additionally, with DISC and DSR, as the size of the response increases, the load on FE remains almost constant while the load increases on BE. Overall, DISC and DSR lead to almost the same results. They significantly offload the CPU load on the FE. **DISC efficiently addresses Q_1, it has almost the same results as DSR in a [FE-BE] architecture**.

Micro-benchmark, [FE-IS *-BE] architecture. As explained earlier, DSR can only support a single bypass (of the FE tier). Accordingly, in a [FE-IS$_1$-IS$_2$-BE] architecture, we compare DISC with its vanilla counterpart (with no bypass at all) and with DSR with a bypass of the FE tier only (initiated by IS$_1$). In Fig. 10, we can see that, with vanilla, considerable CPU time is spent on FE and IS, whereas the load on BE does not increase in the same order when we increase the payload size. This is explained by the fact that FE and IS tiers are both receiving, and sending the payload while the BE is only sending the payload. With DISC, the load on FE and IS tiers remains constant. Further, the load is partially pushed on the BE (as it performs additional tasks such as payload buffering and ACK management). For instance, in the micro-benchmark with a 64KB payload size, DISC offloads FE/IS tiers respectively by 63.4% (FE), 64.3% (IS$_1$), 60.4% (IS$_2$), and increases the CPU load on the BE by 98.8%. With DSR, the load on BE and IS$_2$ is the same as vanilla (nothing changed on these tiers), and offload takes place between IS$_1$ and FE. As depicted in Table 1, for the [FE-IS *-BE] 64KB experiment, the cumulative CPU load across all tiers is 246% for Vanilla, 208% for DSR, and 145% for DISC. Thus, a reduction of 41% of the load with DISC compared to vanilla and 30.3% compared to DSR. **In a [FE-IS *-BE] architecture, DISC also efficiently addresses Q_1, it drastically offload load from FE/IS tiers.**

SpecWeb and SpecMail. As shown in Fig. 11, we observed the same trends as above. Overall, DISC achieves a global reduction of the load, about 41% for SpecWeb and 36% for SpecMail (Table 1). **DISC addresses Q_1, we observe a significant offload of the FE/IS tiers, and a limited increase of the load on the BE.**

Scenario	Vanilla	DSR	DISC
[FE-BE] 16KB	52	54 (-3.8%)	54 (-3.8%)
[FE-BE] 32KB	67	65 (3%)	64 (4.5%)
[FE-BE] 64KB	111	75 (32.5%)	74 (33.3%)
[FE-IS*-BE] 16KB	136	132 (2.9%)	120 (11.7%)
[FE-IS*-BE] 32KB	173	158 (8.7%)	128 (26%)
[FE-IS*-BE] 64KB	246	208 (15.4%)	145 (41%)
SpecWeb	130	—	76.11 (41.5%)
SpecMail	115	—	73.06 (36.5%)

Table 1: Cumulative CPU load on all tiers for all our scenarios and reduction percentages with respect to Vanilla.

4.2 Impact of DISC on scalability (Q_2)

Setup. To study scalability of [FE-BE] architectures, we generate requests, with a payload size of 32KB, at a growing rate, i.e., increasing the rate by 2K req/s every 3 seconds. We run this scenario while varying the number of cores on the FE and BE tiers. Initially, each tier is allocated 2 cores. Upon reaching saturation, the number of cores for the saturated tier doubles.

Results, vanilla settings. In Fig. 4-top, using the vanilla settings, we notice that the front-end (FE) is the first tier to reach saturation, primarily because of the load (pressure) propagated from the back-end (BE). It validates the initial findings

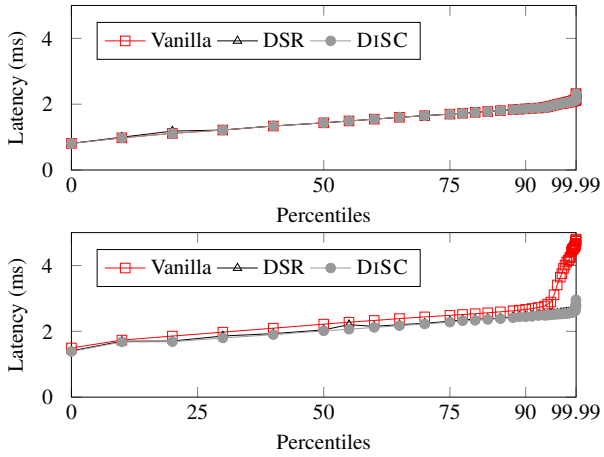

Figure 12: Latency percentiles for the micro-benchmark with the [FE-BE] (top) and [FE-IS$_1$-IS$_2$-BE] (bottom) setups.

presented in §2.2. As soon as the FE saturates, the average request latency raises up, and the doubling of the number of cores is triggered. It appears that with only 2 cores on the FE and BE (on the left side of each sub-figure), tiers achieve a rate of 18K req/s without latency degradation. Having twice the number of cores enables at most a rate of 22K req/s.

Results, DISC settings. In Fig. 4-bottom, using the DISC settings, we observe a notably reduced load on the front-end (FE) in comparison to the vanilla settings. Interestingly, the first tier to reach saturation is the back-end (BE), leading to the increase of: (i) the average latency and therefore (ii) the number of cores (x2). We can observe that with DISC, the tiers achieve a rate of 26K req/s with 2 cores, reaching a plateau at 30K req/s (due to the saturation of the network adapter) with 4 cores.

Compared to the vanilla settings, DISC achieves a significantly higher rate with the same amount of resources: with 2 cores, a rate of 18K for vanilla and a rate of 26K for DISC (an improvement of 45%).

DISC successfully addresses Q_2 by effectively mitigating the backpressure problem. This results in a reduction of the load on tiers preceding the BE, ultimately promoting scalability in the last tiers of a tier chain.

4.3 Impact of DISC on latency (Q_3)

Setup. As previously, the evaluations of latency have been realized with both the micro- and the macro-benchmarks (i.e., SpecWeb and SpecMail). Additionally, we performed evaluation with and without IS tiers involved.

Results, micro-benchmarks. Fig. 12 respectively show the results for zero IS (top) and two IS (bottom) using the micro-benchmarks. From Fig. 12 (top), when no IS is involved, DISC yields nearly identical results to vanilla and

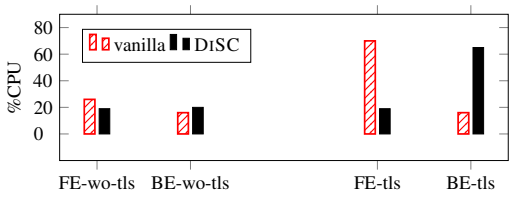

Figure 13: CPU load on each tier in [FE-BE] configuration without and with TLS (micro-benchmark, 32KB payload).

DSR settings. As more IS are involved (Fig. 12 bottom), DISC demonstrates improved latency, with approximately a 0.35ms average difference for 2 IS compared to the vanilla setting and 0.18ms better than DSR. Furthermore, from Fig. 12 (bottom), one can also observe that the tail latency (see the 99.99 percentiles on the figure) is significantly reduced $4.803s \rightarrow 2.959s$. The latency is further improved with more IS tiers, reaching up to $5.71 \times (8s \rightarrow 1.4s)$ for tail latencies with a longer chain of 10 IS (not plotted).

Results, macro-benchmarks. For SpecWeb and SpecMail, we measured that DISC provides a better latency than vanilla by up to 12% in average.

DISC successfully addresses Q_3. It clearly improves latency from the client perspective.

4.4 TLS impact (Q_4)

All the previous experiments were conducted without TLS. To evaluate the impact of TLS, we run the previously used micro-benchmark on a [FE-BE] architecture with and without TLS. Compared to Fig. 9 in §4.1, we reduced the submitted load in order not to saturate CPU with TLS.

As explained in §3.4, when a payload is sent by a BE (when its DP is invoked), the `ssl_session` object is serialized and transmitted from the FE to the DP which deserializes it and uses it for payload encryption. The consequence is that the encryption workload is shifted from FE to BE.

In Fig. 13, we observe the same trend as in previous evaluations. Most of the load is pushed from FE to BE. This is an important benefit for scalability as it prevents FE from becoming a bottleneck. In some way, encryption is part of emission and should be logically handled by the BE without back-propagating the pressure to the preceding tiers.

To answer Q_4, TLS integration in DISC amplifies the mitigation of the backpressure problem.

4.5 Impact on microservices applications (Q_5)

Train Ticket. This macrobenchmark [22] is composed of 43 microservices, which is more complex than the SPEC applications we evaluated above. The microservices communicate with each other via gRPC. An external client request triggers the execution of a specific chain of microservices. The chains vary in length and may involve different sizes of final

data, generated by the database (MySQL). For example, the request corresponding to page `client_adsearch` generates very little final data, which would be irrelevant for DISC. Conversely, the requests triggered by `client_order_list` and several other pages[2] are likely to pull a large amount of final data from MySQL. For our experiments, we focused on the microservices involved in these requests. To illustrate the evaluation of DISC, we use the `client_order_list` request, which involves 4 microservices: `ui` (FE), `gateway` (IS_1), `order` (IS_2), and `mysql` (BE). We keep the default configuration for each service. An important detail to understand the results is that the `gateway` implements traffic shaping, calibrated at 20 req/s.

The modifications made to the application to integrate DISC primarily consisted of extending the SQL language to signal when the return path of a microservice can be bypassed. We also extended the MySQL JDBC driver [9] to handle this SQL extension and implement DISC's DP component.

For this experiment, we use virtual machines (VMs), unlike the previous experiments, which were conducted on bare-metal. This allows DISC to be evaluated in typical application execution environments. Each service is deployed in a separate VM (2 vCPU and 2GB of memory), resulting in a total of 43 VMs. To simplify the interpretation of results, we opted for a setup without microservice replicas. In this virtualized environment, for the `MySQL` VM, the virtual network interface (virtio) is configured with multiple transmission queues, with one queue specifically dedicated to XDP. Using wrk2, we launched 400 clients, each requesting a payload size of 32KB.

Fig. 14 shows the results. We observe that DISC does not significantly reduce the CPU load on the `gateway` compared to other microservices. This is because the `gateway` applies rules to throttle incoming requests upon reception (traffic shaping), which adds computational overhead. To support this, we conducted experiments with a 0B payload and we observed that the CPU load on the `gateway` remains high, indicating that it is primarily driven by the computations from its traffic shaping module. The load on the `gateway` increases with DISC, compared to the vanilla setup, when the payload is 0B because DISC allows the application to receive more traffic, resulting in more traffic shaping processing on the `gateway`. DISC shifts the CPU load from the `order` service (49% reduction w.r.t. vanilla), while moderately increasing it on the MySQL BE (+18.5%).

We also observe that DISC improves both latency and throughput. Average latency decreases by 74.1%, from 3.57 to 0.928 seconds, and throughput increases by 39.8%, from 635.80 to 889.20 req/s.

Social Media. Due to a lack of space, we only provide a brief summary of the results for this macro-benchmark. More details on the benchmark, setup and experiments, are available in Appendix B. Similarly to Train Ticket (§4.5), DISC shifts

[2] `admin`, `admin_travel`, `admin_user`, `client_consign_list`, `client_enter_station`, and `client_ticket_collect`.

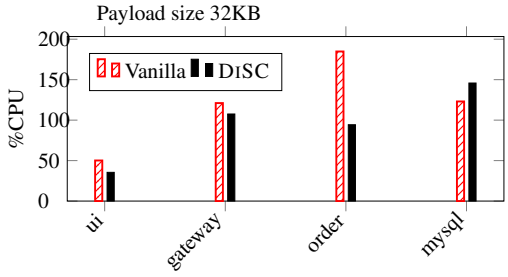

Figure 14: CPU load on the Train-ticket microservices triggered by a `client_order_list` request. `ui` and `mysql` correspond to FE and BE, respectively.

the CPU load from frontend to backend tiers. Compared to vanilla, DISC offloads the frontend tiers by up to 41.7% and increases the backend tier by up to 48.8%. Moreover, DISC reduces the average latency by up to 66.7% by dropping it from 510ms to 221ms, while increasing the throughput by 21.9%, from 80355 req/s to 98009 req/s.

DISC successfully addresses Q_5: its benefits are applicable to coarse-grained as well as to microservices applications.

5 Conclusion

This paper tackles the backpressure problem, which compromises both: (i) the independence of tiers regarding performance, and (ii) the scalability of multi-tier architectures. We presented the drawbacks of state-of-the-art solutions and identified six requirements (flexible, protocol agnostic, transparent, pervasive, consistent, TLS support) that a solution should meet to efficiently address this problem in a holistic manner. We introduced our solution, DISC, which meets all these requirements. With a prototype built atop TCP, we demonstrated that DISC significantly: (i) reduces the CPU load, and (ii) improves tail latencies on heavy-load scenarios.

Note that, like CRAB [20], DISC relies on IP spoofing, which is generally not possible for IaaS tenants. Thus, in the context of public cloud platforms, DISC can only be deployed by cloud providers to complement (or replace) their existing offerings (load balancers and managed applications).

Acknowledgements

We thank our shepherd, Ankit Bhardwaj, and the anonymous reviewers for their insightful comments. Théo Boury, Maxime Bruno, and Adrien Vannson contributed to early versions of the DISC implementation. This work was supported by the French *Agence Nationale de la Recherche* under the ANR projects "PicNic" (ANR-20-CE25-0013) and "PEPR DiVa" (ANR-23-PECL-0004), by the CNRS "MLSN2" International Research Project, and by Inria "Défi OS".

References

[1] CloudLab. https://www.cloudlab.us/ (Accessed on 2023-04-12), 2023.

[2] Connection Migration in QUIC. https://datatracker.ietf.org/doc/html/draft-tan-quic-connection-migration-00 (Accessed on 2023-04-16), 2023.

[3] Dovecot. https://www.dovecot.org/ (Accessed on 2023-12-06), 2023.

[4] HAProxy: The Reliable, High Performance TCP/HTTP Load Balancer. https://www.haproxy.org/ (Accessed on 2023-03-20), 2023.

[5] NGINX. https://www.nginx.com/ (Accessed on 2023-03-23), 2023.

[6] wolfSSL Embedded TLS Library. https://www.wolfssl.com/ (Accessed on 2023-03-20), 2023.

[7] Zimbra email & collaboration. https://www.zimbra.com/ (Accessed on 2023-10-18), 2023.

[8] Apache Thrift. https://thrift.apache.org/ (Accessed on 2024-07-23), 2024.

[9] MySQL Community Downloads - Connector/J. https://dev.mysql.com/downloads/connector/j/ (Accessed on 2024-06-18), 2024.

[10] Carl Alexander. A Look At The Modern WordPress Server Stack. https://www.smashingmagazine.com/2016/05/modern-wordpress-server-stack/ (Accessed on 2024-05-05), 2016.

[11] Mohit Aron, Darren Sanders, Peter Druschel, and Willy Zwaenepoel. Scalable Content-Aware Request Distribution in Cluster-Based Networks Servers. In *Proceedings of the USENIX Annual Technical Conference*, USA, 2000. USENIX Association.

[12] Standard Performance Evaluation Corporation. SpecMail2009. https://www.spec.org/mail2009/ (Accessed on 2023-12-06).

[13] Standard Performance Evaluation Corporation. SpecWeb2009. https://www.spec.org/web2009/ (Accessed on 2023-12-06).

[14] Colin Dixon, Hardeep Uppal, Vjekoslav Brajkovic, Dane Brandon, Thomas Anderson, and Arvind Krishnamurthy. ETTM: A Scalable Fault Tolerant Network Manager. In *8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11)*, Boston, MA, March 2011. USENIX Association.

[15] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Colen, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvinsky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '19, page 3–18, New York, NY, USA, 2019. Association for Computing Machinery.

[16] Will Glozer. wrk - a HTTP benchmarking tool . https://github.com/wg/wrk (Accessed on 2023-12-06).

[17] Yutaro Hayakawa, Michio Honda, Douglas Santry, and Lars Eggert. Prism: Proxies without the Pain. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 535–549. USENIX Association, April 2021.

[18] Guerney Hunt, Erich Nahum, and John Tracey. Enabling Content-Based Load Distribution for Scalable Services. Technical Report TR-97. IBM T.J. Watson Research Center, 1997.

[19] Matt Klein. Introduction to Modern Network Load Balancing and Proxying. https://blog.envoyproxy.io/introduction-to-modern-network-load-balancing-and-proxying-a57f6ff80236/ (Accessed on 2024-05-05), 2017.

[20] Marios Kogias, Rishabh Iyer, and Edouard Bugnion. Bypassing the Load Balancer without Regrets. In *Proceedings of the 11th ACM Symposium on Cloud Computing*, SoCC '20, page 193–207, New York, NY, USA, 2020. Association for Computing Machinery.

[21] Marios Kogias, George Prekas, Adrien Ghosn, Jonas Fietz, and Edouard Bugnion. R2P2: Making RPCs first-class datacenter citizens. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 863–880, Renton, WA, July 2019. USENIX Association.

[22] Fudan University Software Engineering Laboratory. Train Ticket - A Benchmark Microservice System. https://github.com/FudanSELab/train-ticket/tree/master (Accessed on 2024-05-05), 2018.

[23] Adam Langley, Alistair Riddoch, Alyssa Wilk, Antonio Vicente, Charles Krasic, Dan Zhang, Fan Yang, Fedor Kouranov, Ian Swett, Janardhan Iyengar, Jeff Bailey, Jeremy Dorfman, Jim Roskind, Joanna Kulik, Patrik

Westin, Raman Tenneti, Robbie Shade, Ryan Hamilton, Victor Vasiliev, Wan-Teh Chang, and Zhongyi Shi. The QUIC Transport Protocol: Design and Internet-Scale Deployment. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, SIGCOMM '17, page 183–196, New York, NY, USA, 2017. Association for Computing Machinery.

[24] Parveen Patel, Deepak Bansal, Lihua Yuan, Ashwin Murthy, Albert Greenberg, David A. Maltz, Randy Kern, Hemant Kumar, Marios Zikos, Hongyu Wu, Changhoon Kim, and Naveen Karri. Ananta: Cloud Scale Load Balancing. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM*, SIGCOMM '13, page 207–218, New York, NY, USA, 2013. Association for Computing Machinery.

[25] Carlo Puliafito, Luca Conforti, Antonio Virdis, and Enzo Mingozzi. Server-Side QUIC Connection Migration to Support Microservice Deployment at the Edge. *Pervasive and Mobile Computing*, 83:101580, 2022.

[26] Aigars Silkalns. WordPress Statistics: How Many Websites Use WordPress in 2024? https://colorlib.com/wp/wordpress-statistics/ (Accessed on 2024-05-05), 2024.

[27] Alex C. Snoeren, David G. Andersen, and Hari Balakrishnan. Fine-Grained Failover Using Connection Migration. In *3rd USENIX Symposium on Internet Technologies and Systems (USITS 01)*, San Francisco, CA, March 2001. USENIX Association.

[28] Lluís Vilanova, Lina Maudlej, Shai Bergman, Till Miemietz, Matthias Hille, Nils Asmussen, Michael Roitzsch, Hermann Härtig, and Mark Silberstein. Slashing the Disaggregation Tax in Heterogeneous Data Centers with FractOS. In *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys'22)*, EuroSys '22, page 352–367, New York, NY, USA, 2022. Association for Computing Machinery.

[29] Stephanie Wang, Eric Liang, Edward Oakes, Ben Hindman, Frank Sifei Luan, Audrey Cheng, and Ion Stoica. Ownership: A Distributed Futures System for Fine-Grained Tasks. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 671–686. USENIX Association, April 2021.

[30] Ziqi Wei, Zhiqiang Wang, Qing Li, Yuan Yang, Cheng Luo, Fuyu Wang, Yong Jiang, Sijie Yang, and Zhenhui Yuan. QDSR: Accelerating Layer-7 Load Balancing by Direct Server Return with QUIC. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 715–730, Santa Clara, CA, July 2024. USENIX Association.

Appendices

The appendices are organized as follows: Appendix A provides implementation details regarding the implementation of the different components of DISC, Appendix B provides additional details regarding the Social Media benchmark, setup and experiments (discussed in §4.5).

A Implementation details

DISC is implemented with a kernel module (kHook, kvs), two user-level modules (ackSender and DP) and a set of L7 hooks to act at the L7 layer (i.e., feHook, isHook, beHook). kHook relies on BPF (extended Berkeley Packet Filter) for intercepting and adapting packets at the L4 layer. kvs is implemented with BPF maps (a generic storage system for sharing data between the kernel and the user space). L7 hooks can be implemented with patches or application extensions when such a support is provided (e.g., NGINX allows the definition of filters). The overall design of DISC is depicted in Figure 5. The implementation details of each component is given below.

kHook. The kHook is activated on each server of the datacenter hosting a tier that may act as FE. kHook is composed of three interceptors called `in_tc`, `out_tc`, and `in_drv`. Both `in_tc` and `out_tc` are implemented in the TC (Traffic Control) layer while `in_drv` is integrated in the NIC driver. kHook could be fully implemented at the TC level, but we introduced `in_drv` for performance reasons (to prevent needless walks in the protocol stack). `in_tc` intercepts incoming TCP connections and initializes kvs with the characteristics of the established connection. All other incoming packets are handled by `in_drv`.

`in_drv` intercepts all packets received from the client after connection. Such packets may either include data or not, and must include an ack. If the packet includes data, then data is transmitted to the stack of FE. But anyway (whether the packet includes data or not), acks must be handled, so that they are consistently (regarding sequence numbers SN) sent to the stack of FE or to DP (which emitted the payload). If there were not any bypass yet, packets are transmitted to the FE stack as is. As soon as a bypass occurred, SN included in acks have to be translated as depicted in §3.3. A map of the SN that were emitted (returned to the client) by the FE stack and the DP is registered in the kvs. This map includes emitted SN ranges and the IP addresses of the emitting servers. Both are kept in the kvs as long as they are not acknowledged by EC. A SN in a received ack that acknowledges data from a range generates an ack which is sent to the emitting server of the range (FE or DP) with the highest acknowledged SN from the range (notice that several ranges may be acknowledged by an ack). If the generated ack is transmitted to FE, the SN is translated back into the SN as sent by the FE. If the generated ack is to be sent to the DP, it is actually stored in kvs and consumed by the ackSender process.

Regarding out_tc, it translates SN sent by FE to the client, taking into account the global SN state (see §3.3) that is returned when DP is invoked (for sending a payload).

lib_kHook. This library provides an API for instantiating kHook within the kernel and for accessing kvs. It provides: (i) setup_khook() to instantiate the hook, (ii) poll_ack() to poll acks from kvs to be sent to DP by ackSender, and (iii) register_range() to register a packet range sent by BE.

register_range() is called by feHook after the reception of the response header that includes in its DISC-PROT header the ids (identifier) of the payload, its size and IP of the BE (location of DP). Those parameters are passed to register_range(), which registers in kvs the packet range associated with the payload and information for handling acks for these packets (IP of BE and ids). register_range() also receives as parameters the SN associated with the registered packet range (deduced from the global SN state and the size of the payload) and also the IP address and port of the client. Notice that register_range() also receives a callback function, invoked by kHook when the client receives the full payload (thus allowing to handle protocol footers, e.g., in the case of IMAP).

lib_comm. This library implements an API for communication between DISC components located on different machines. It provides two key functions: (i) start_transmit() to remotely invoke a DP, and (ii) forward_ack() to forward acks to a DP. start_transmit() takes as parameters the IP of BE and ids of the payload, address of EC (IP and port) and the global SN state to be used for sending the payload.

ackSender. ackSender is a process started on each FE. It relies on poll_ack() from lib_kHook and forward_ack() from lib_comm to extract acks from kvs and send them to DP. Since acks can be any of the following ACK, SACK, DUP ACK and ZERO WINDOW, DP can therefore manage the buffering of the payload (freeing buffers), the re-emission of lost packets and its emission window.

feHook. feHook calls register_range() after the reception of the response header to store in kvs information about the emitted payload. It then calls start_transmit() to request the emission of the payload from DP.

beHook and DP. The implementation of DP relies on two modules called ctr_msg_manager and payload_sender. Both modules include a thread for handling to incoming requests. ctr_msg_manager handles requests issued by FE when either: (i) an ack is forwarded *via* forward_ack(), or (ii) the emission of a payload is requested *via* start_transmit()). payload_sender handles the emission of payloads (i.e., packets on the connection preceding the FE).

When beHook (which acts at the L7 layer of BE) decides to send a payload with bypass, it asks ctr_msg_manager to create a job that is inserted into a red-black tree structure. This job includes a unique identifier (ids) of the payload, and a data structure representing the content of the payload. In our prototype, the content of the payload in a job can be of two types. In the first case (type 1), the payload content is a buffered stream of data sent by the L7 layer of BE. BE starts filling the buffer and blocks until buffers are freed by received acks. In the second case (type 2), the payload content is a file, meaning that emitted packets are directly read from a file. This is equivalent to the sendfile() Linux primitive, which directly copies data from a file descriptor to a socket descriptor.

When feHook (on FE) calls start_transmit() to request the emission of the payload, ctr_msg_manager receives that request, and retrieves the job identified by ids from the red-black structure. It adds in the job the parameters received in the request (IP address and port of the client, SN associated with the payload, size of the reception window of the client) and asks payload_sender to insert a task in its internal task list (a FIFO list of tasks consumed by the handler thread). A task is a request to emit a data chunk. When starting transmission of a payload, the created task corresponds to the emission of a chunk whose size is the reception window of the client. Then, when ctr_msg_manager receives an ACK, DUP_ACK or SACK, it implies that either a chunk has to be re-sent or that an additional chunk within the payload can be sent. Therefore, ctr_msg_manager handles such acks by adding tasks in the task list. The handling of a task reads the chunk either from the buffer or the file (according to the job) and generates a set of packets on the TCP connection preceding the FE (thus DP is spoofing FE).

B DISC with the Social Media benchmark

We perform additional experiments to show how DISC performs with complex microservices. To meet this aim, we modified the Social Media benchmark from the DeathStarBench benchmark suite [15]. The Social Media benchmark comprises 36 services, primarily written in C, C++, Java, and Node.js, communicating through Thrifts RPC [8]. The storage services run MongoDB with caching services that run Memcached or Redis. A client request queries a load balancer, which distributes a client request across Nginx web servers. The request processing chain length depends on the client's request input. Like for Train Ticket (§4.5), to apply DISC, we modify the storage and caching layers and their connectors (MongoDB, Memcached, and Redis) to obtain the microservices that can be bypassed when receiving a read or write request. Consequently, we reduce the load on the intermediate nodes hosting the bypassed microservices.

We deploy our modified version of the Social Media benchmark where each service runs in VMs that have the same size as the ones from Section 4.5 (2vCPU and 2GB memory) and

use the built-in load generator (built atop wrk) to evaluate the impact of DISC. Concretely, we use the load generated to simulate 1000 clients that read randomly previously registered posts for 600 seconds. 1000 clients is enough to stress the frontend tiers cores up to 70% overall. Each client triggers a request every 10ms. We chose this scenario because it is among the scenarios with the longest request processing chain. The input payload size for such a request is 4KB, and the response payload size depends on the requested post, which varies from 64KB to 8.2MB. During the experiment, we record the CPU usage on all microservice nodes. Figure 15 reports the CPU usage on the frontend tier `LB`, the intermediate (IS) tiers `Media frontend, Nginx` and the backend tier `MongoDB`. The caches hit ratio is, on average, 2.8%. Thus, we do not focus on the CPU load on the caching nodes.

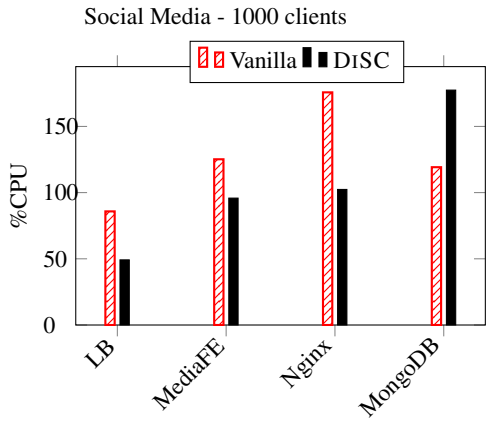

Figure 15: CPU load on the main microservices triggered during the execution of the read post requests performed by 1000 clients during 600s. `LB` correspond to the `FE`, `Media frontend (MediaFE), NGINX` correspond to `IS`, and `MongoDB` correspond to the `BE`.

Similarly to Train Ticket (§4.5), DISC shifts the CPU load from frontend to backend tiers. Concretely, DISC offloads the frontend tiers by up to 41.7% and increases the backend tier by up to 48.8%. Moreover, DISC reduces the average latency by up to 66.7% by dropping it from 510ms to 221ms, while increasing the throughput by 21.9%, from 80355 req/s to 98009 req/s. Thus, DISC efficiently shifts load from FE to BE, improving overall end-end latency and throughput compared to vanilla.